



Order Book Analyzer – Status Report 2

FINTECH 512 - Software Engineering for FinTech

Full team of five members for the group project (section 1)



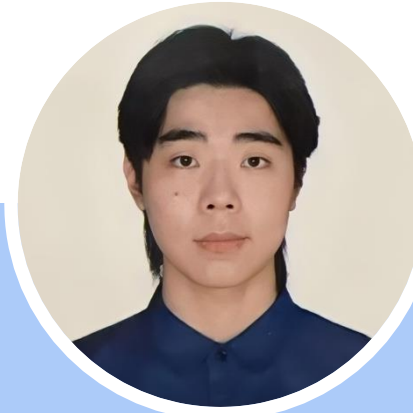
Rebeca Hechtman
NetId: rh398



Mike Lin
NetId: hl561



Mory Shi
NetId: cs836



Loki Luo
NetId: lz294



Alan Song
NetId: ys501

Agenda

- 01.** Project Ideation
- 02.** GitLab Repository and Issues
- 03.** Updated Design
- 04.** Status, Review and Planning

Project Ideation

Project's 'elevator pitch'



Bloomberg of Prediction Markets



Prediction markets, where people bet real money on the outcomes of future events (like whether Bitcoin will hit \$110,000 this year), are exploding in popularity. But making sense of all that betting activity can be frustrating. The raw data behind these markets is messy, disorganized, and hard to easily analyze.

Order Book Analyzer cuts through that noise. Think of us as the Bloomberg for prediction markets. We take massive amounts of complicated, second-by-second betting data and translate it into clear, powerful dashboards and insights. Instead of wrestling with messy data, our platform gives you the exact trends and historical information you need to see exactly where the smart money is going.

Project Scope Disclaimer: This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1). Future market expansions remain under evaluation based on technical complexity and timeline constraints.

The Lean Canvas

Designed for:

Order Book Analyzer

Designed by:

Team Hotel

Date:

03/16/2026

Version:

2.0

Problem  1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution  A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop.  High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage  1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments  - Mid and small size Hedge Funds - Sophisticated algo traders who wants to trade in prediction market
Existing Alternatives  - Eagle Alpha - Dune Analytics - DIY engineering - Basic Polymarket Dashboards	Key Metrics  Growth: # of users - Retention - API call volumes - Daily Active User System Health: - Data Quality Score	High-Level Concept  - Yahoo Finance for Prediction Market - Bloomberg for Prediction Market	Channels  - Social Media (LinkedIn, X, Reddit, Red Note) - Organic Growth - Search Engine - Ads	Early Adopters  Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.
Cost Structure  Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.		Revenue Streams  Revenue Model: Freemium <u>Freemium Model:</u> Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. <u>Key Account (KA) Annual Licenses:</u> High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. <u>Value-Added Services (Overage):</u> Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.		

Changes we've made since the project started

The Lean Canvas		Designed for:	Designed by:	Date:	Version:
		Order Book Analyzer	Team Hotel	03/16/2026	2.0
Problem 1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop. High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage 1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments 1. Mid and small size Hedge Funds 2. Sophisticated algo traders who wants to trade in prediction market	
Existing Alternatives 1. Eagle Alpha 2. Dune Analytics 3. DIY engineering 4. Basic Polymarket Dashboards	Key Metrics Growth: 1. # of users 2. Retention 3. API call volumes 4. Daily Active User System Health: 1. Data Quality Score	High-Level Concept 1. Yahoo Finance for Prediction Market 2. Bloomberg for Prediction Market	Channels 1. Social Media (LinkedIn, X, Reddit, Red Note) 2. Organic Growth 3. Search Engine 4. Ads	Early Adopters Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.	
Cost Structure Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.		Revenue Streams Revenue Model: Freemium Freemium Model: Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. Key Account (KA) Annual Licenses: High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. Value-Added Services (Overage): Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.			

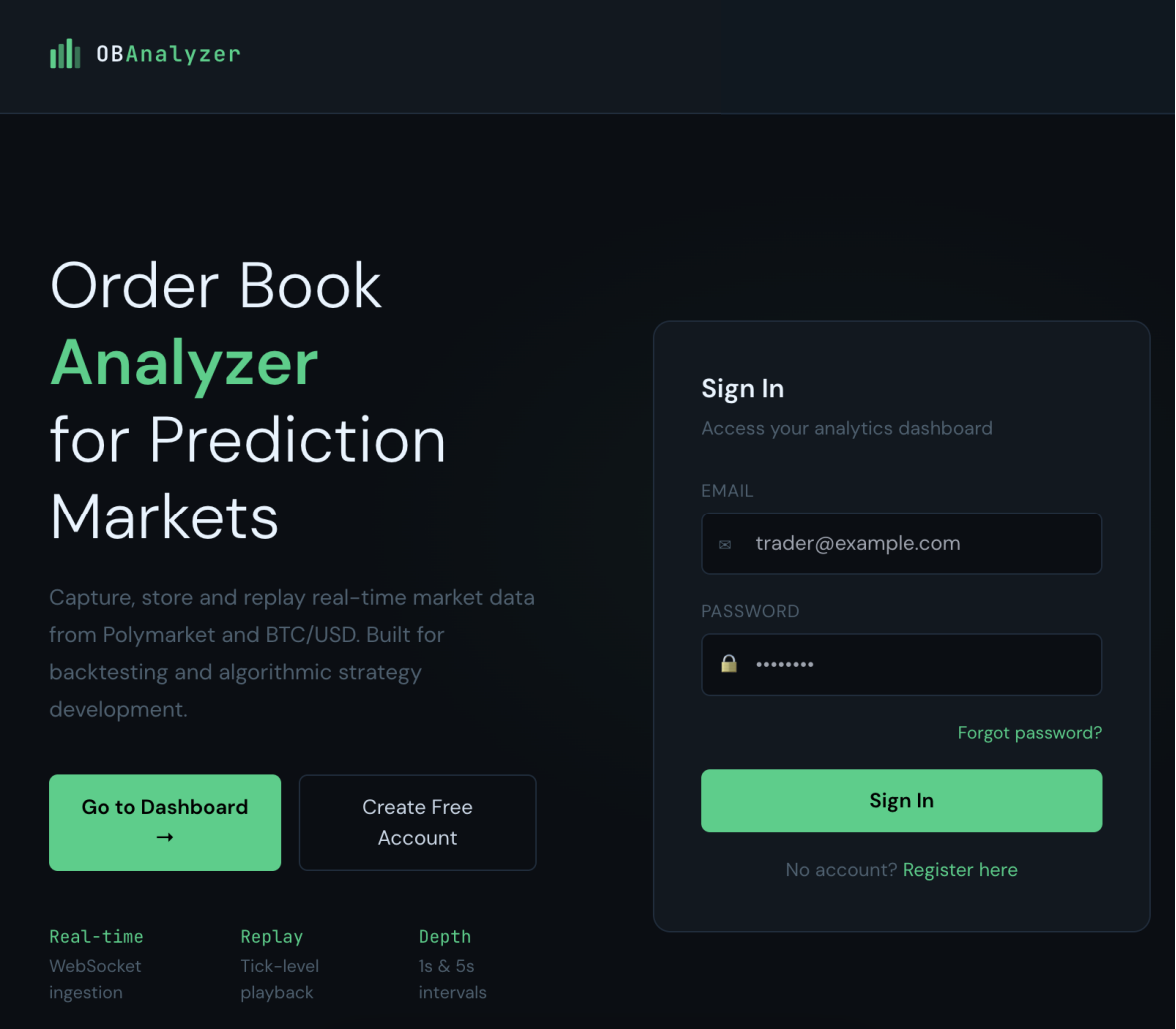
Lean Canvas is adapted from The Business Model Canvas (www.businessmodelgeneration.com/canvas). PowerPoint Implementation by: Neos Chronos Limited (<https://neoschronos.com>). License: CC BY-SA 3.0

Scope Restriction:

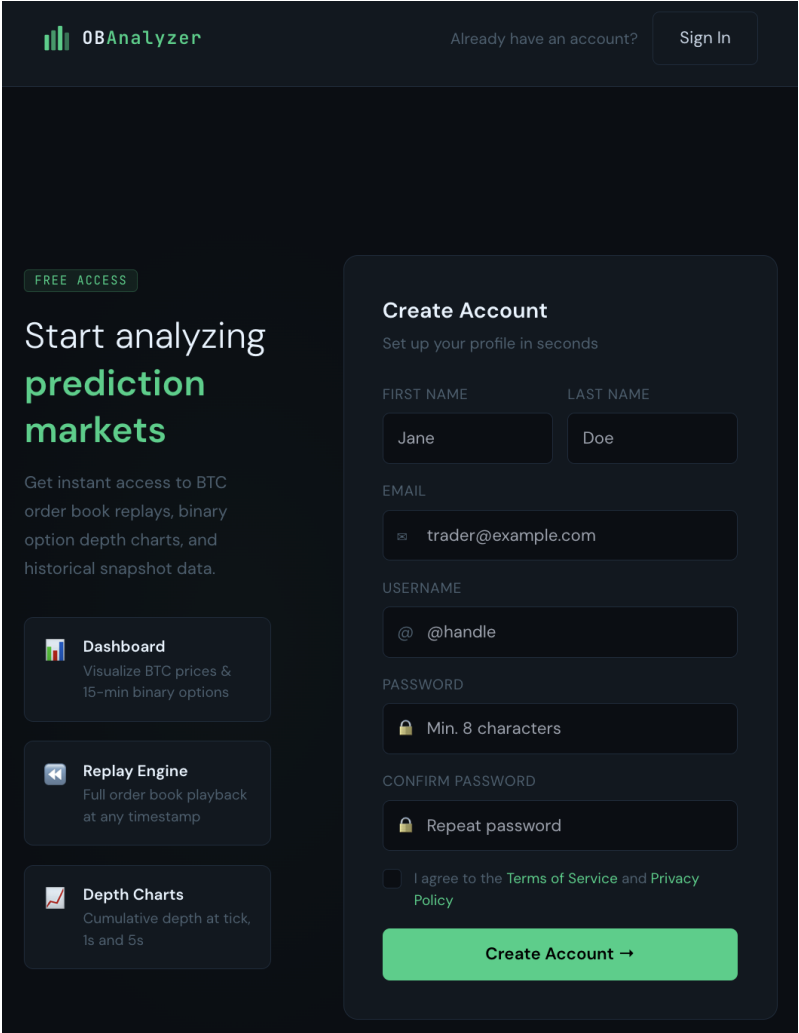
- This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1).
- Future market expansions remain under evaluation based on technical complexity and timeline constraints.

Prototype Screens

1- Home Page

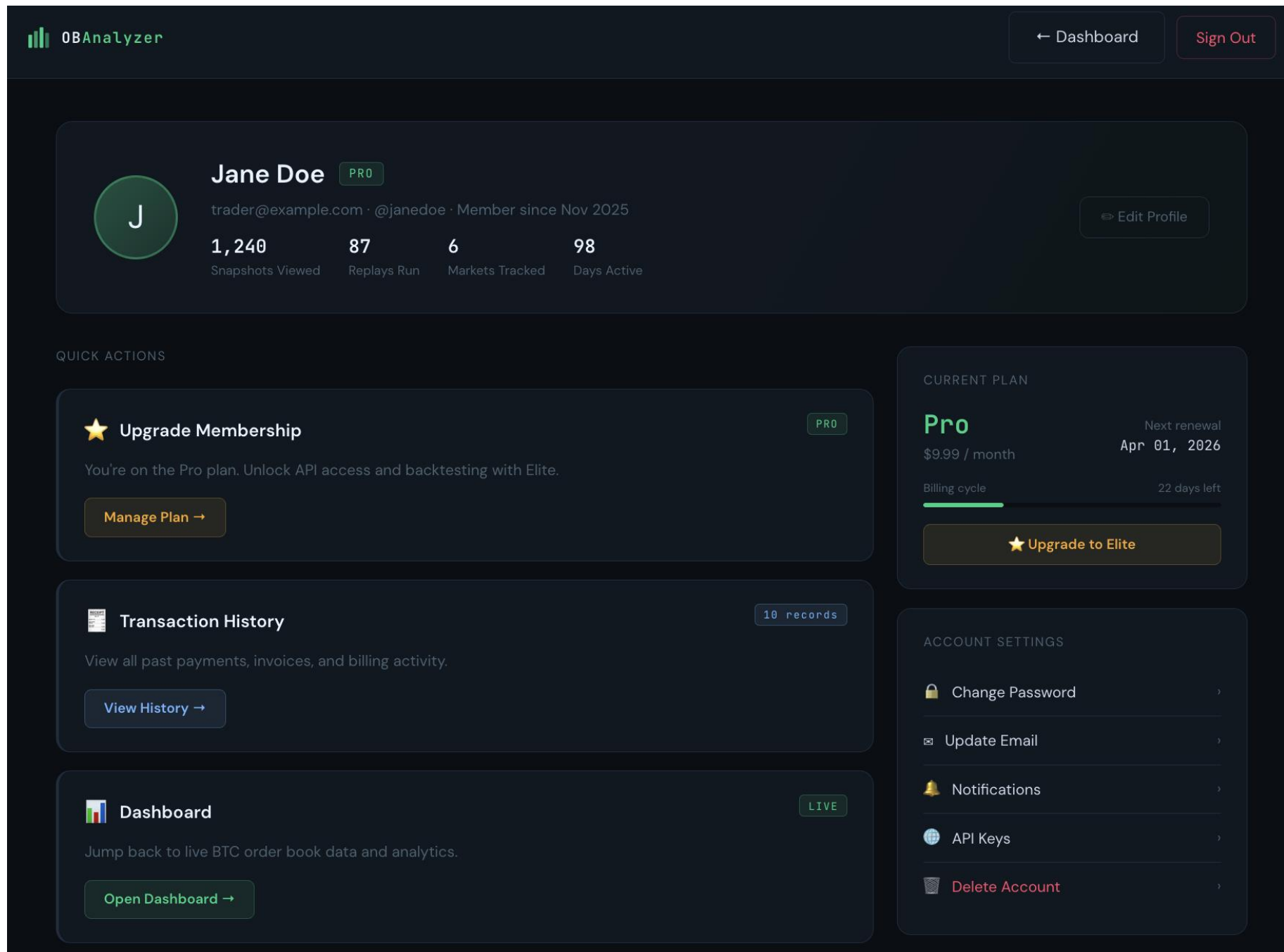


2- New User



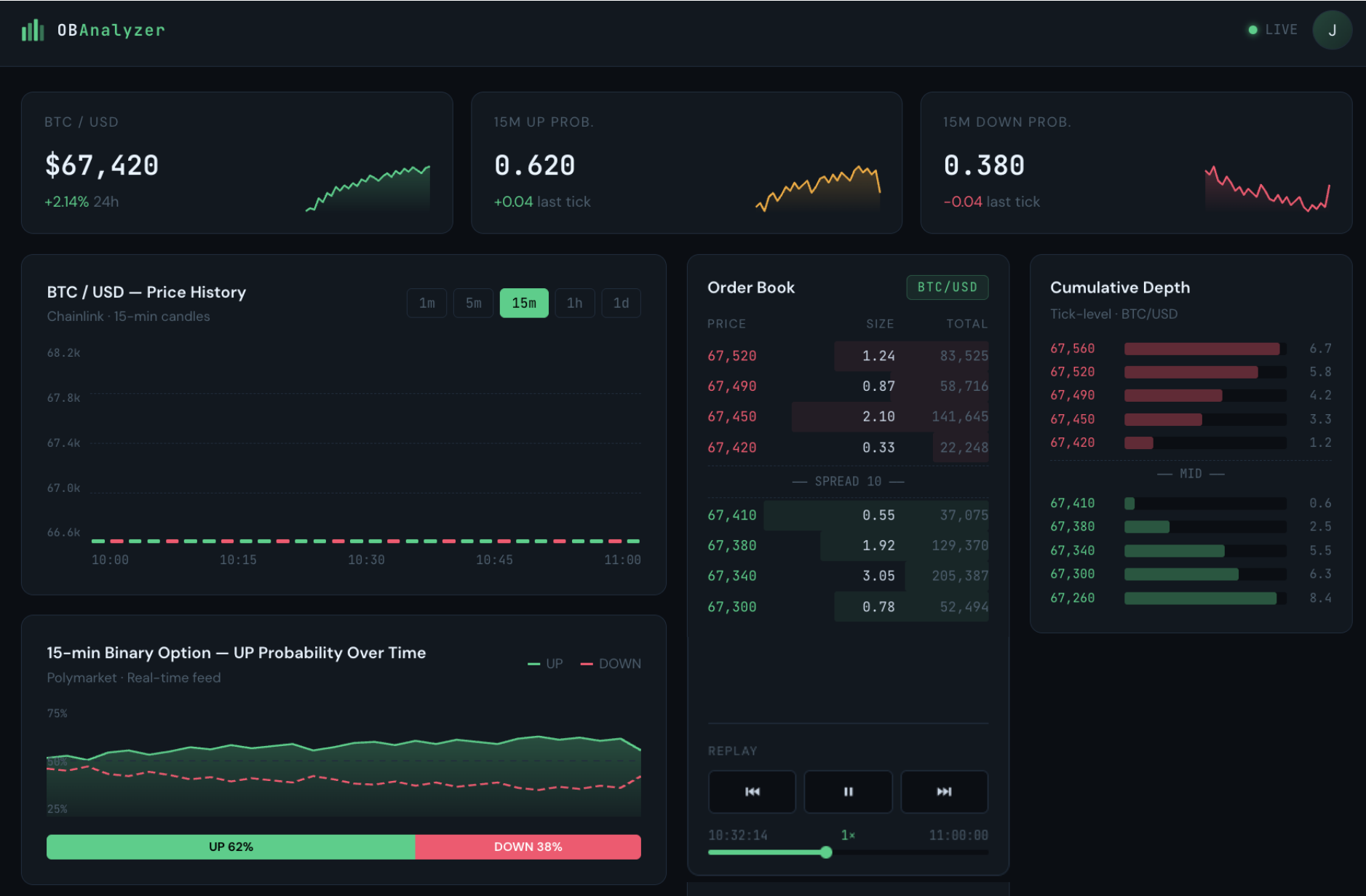
Prototype Screens

3- Profile



Prototype Screens

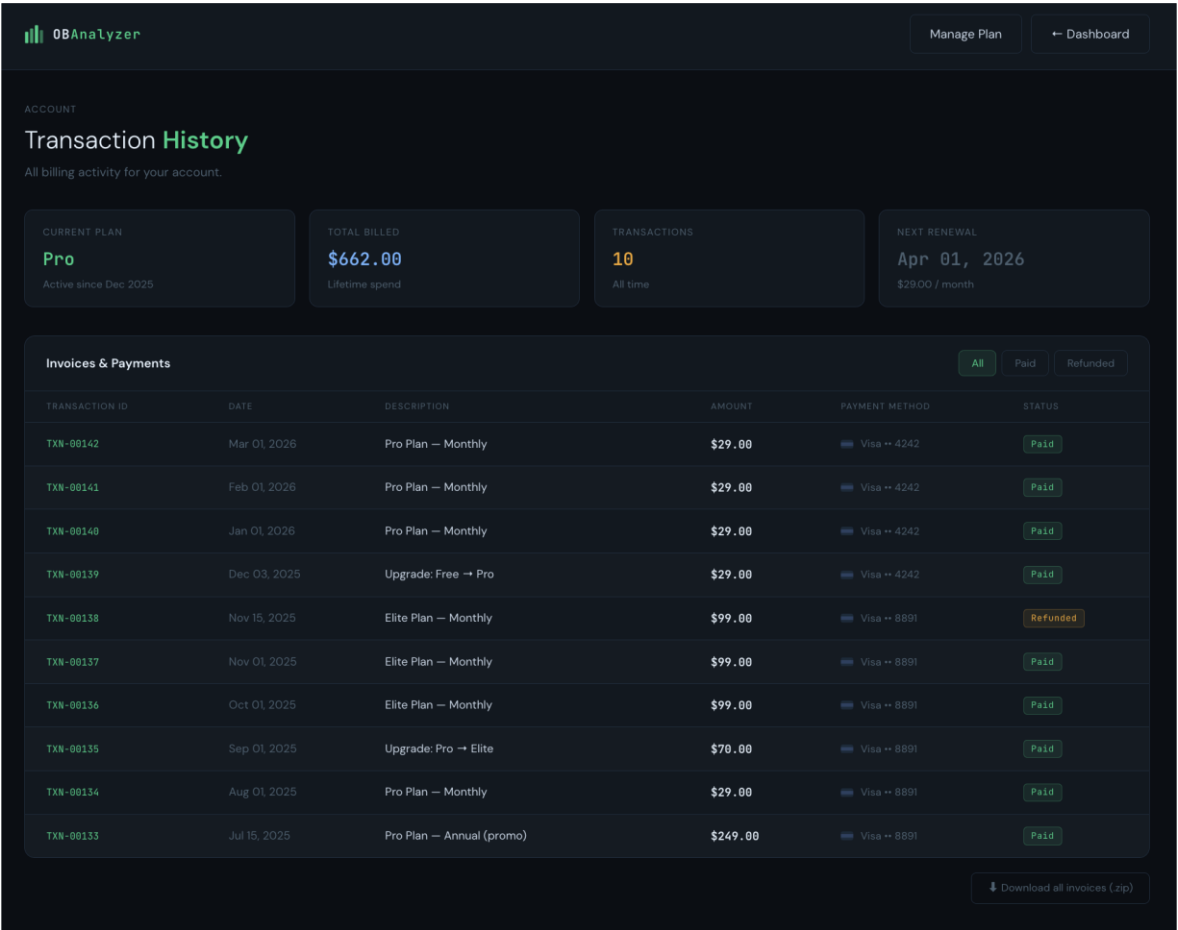
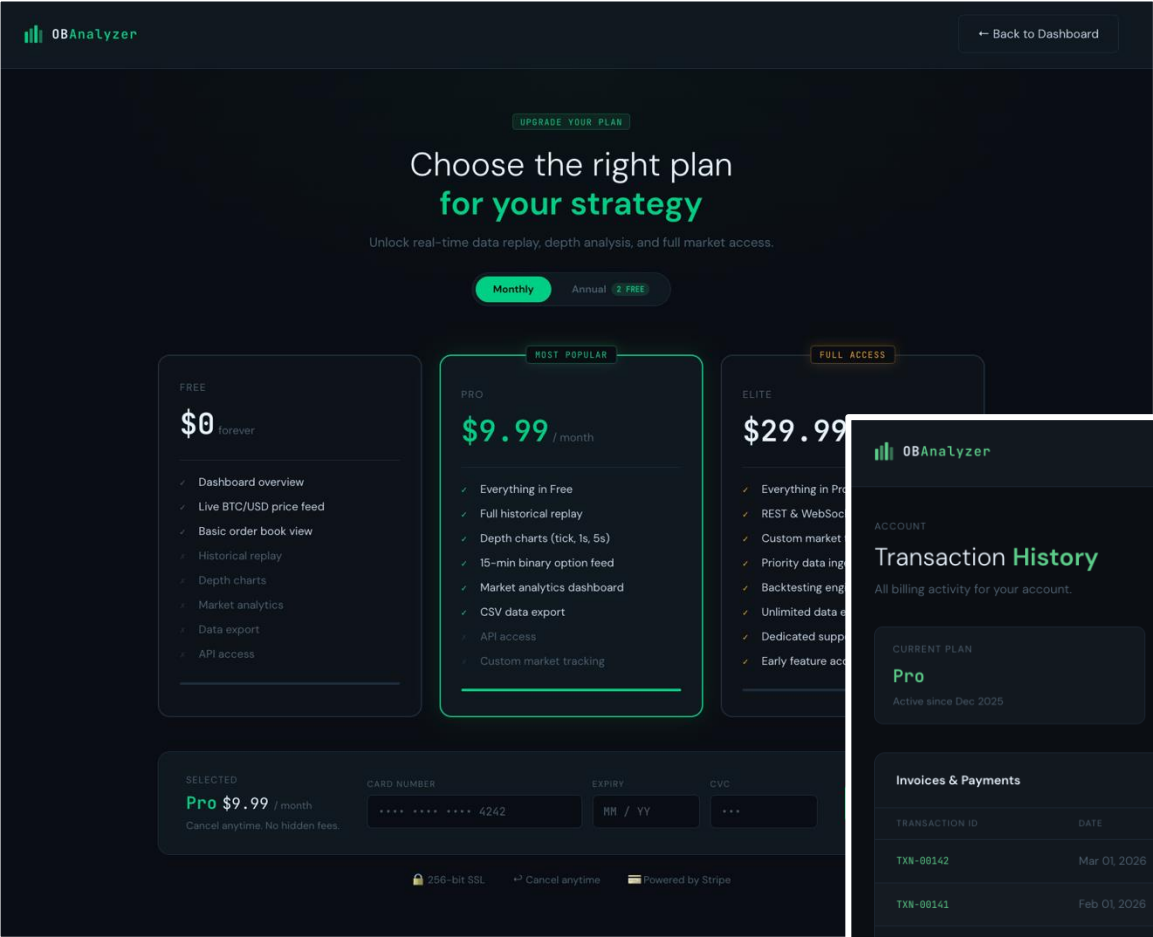
4-Dashboard



Prototype Screens

5- Manage Plan

6- Transaction History



GitLab Repository and Issues

Team Project GitLab Repository

https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#

The screenshot shows a web browser displaying the GitLab interface for a repository named 'pred_market_data_platform' under the 'team-hotel' namespace. The URL in the address bar is 'gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#'. The left sidebar contains a 'Project' section with a link to the repository and a 'Pinned' section with links to 'Issues' and 'Merge requests'. Below these are links for 'Manage', 'Plan', 'Code', 'Build', 'Secure', 'Deploy', 'Operate', 'Monitor', 'Analyze', and 'Settings'. At the bottom of the sidebar are links for 'What's new' and 'Help'. The main content area shows the repository name 'pred_market_data_platform' with a 'Code' button. Below this is a message stating 'The repository for this project is empty' and instructions to clone the repository or upload files. There are sections for 'Command line instructions', 'Configure your Git identity' (with 'Local' and 'Global' tabs), 'Git local setup' (with instructions to configure identity locally), 'Add files' (with instructions to push files using SSH or HTTPS), and 'Create a new repository' (with a 'git clone' command). On the right side, there is a 'Project information' section with an 'Invite your team' button and a list of actions: 'Upload File', 'New file', 'Add README', 'Add LICENSE', 'Add CHANGELOG', 'Add CONTRIBUTING', 'Set up CI/CD', 'Add Wiki', and 'Configure Integrations'. At the bottom right, it says 'Created on March 06, 2026'.

team-hotel / pred_market_data_platform

Search or go to...

Project

- pred_market_data_platform
- Pinned
- Issues
- Merge requests
- Manage
- Plan
- Code
- Build
- Secure
- Deploy
- Operate
- Monitor
- Analyze
- Settings

What's new

Help

Collapse sidebar

team-hotel / pred_market_data_platform

pred_market_data_platform

Code

The repository for this project is empty

To get started, clone the repository or upload some files.

Command line instructions

You can also upload existing files from your computer using the instructions below.

Configure your Git identity

Get started with Git and learn how to configure it.

Local Global

Git local setup

Configure your Git identity locally to use it only for this project:

```
git config --local user.name "rebecca.hechtman@duke.edu"
git config --local user.email "rh398@duke.edu"
```

Add files

Push files to this repository using SSH or HTTPS. If you're unsure, we recommend SSH.

SSH HTTPS

How to use SSH keys?

Create a new repository

```
git clone git@gitlab.oit.duke.edu:team-hotel/pred_market_data_platform.git
```

Project information

Invite your team

Add members to this project and start collaborating with your team.

Invite members

Upload File

- New file
- Add README
- Add LICENSE
- Add CHANGELOG
- Add CONTRIBUTING
- Set up CI/CD
- Add Wiki
- Configure Integrations

Created on

March 06, 2026

Project Requirements – Organization and Ideation

Legend:

- Done
- In progress
- To be done

1. Organization

1. W1: List the requirements
2. W2: Refine the requirements - Rebeca

2. Gitlab

1. W1: Create Gitlab
2. W2: Update the issues (+ improve wording) – Alan
3. W2: Understand Gitlab SAST Tooling - Mory

3. System Architecture diagrams (C4) – Mory

1. W1: Create System Architecture
2. W2: Create using C4 model – Mory
3. Refine diagrams

4. Virtual Machines (current version of site running on a Duke VCM)

1. W1: Apply VCM
2. W2: Share the VCM port with the group - Mike
3. Deploy the app (Gitlab -> virtual machine)
4. Automatically starting

5. Security Analysis - DFD threat model

Create a Data Flow Diagram (DFD) for your BigBucks system, identify assets, actors, and threats, and perform a STRIDE analysis of DFD

1. W2: 1st overview draft - Loki
2. Refinement and detailing

6. Users and Administrative Functionality:

1. W1: initial user's type ideation
2. W2: Admin's ideation
3. Detailing of user
4. Detailing of admin

7. Data service

1. W1: data design ideation
2. W1: data flow ideation (APIs)
3. W2 and W3: testing ideation – Mike

8. Full stack

1. W2-W3: Data design of user and admin ideation - Alan
2. W2-W3: ideation of testing - Loki

9. Charting (3 or more + custom interaction)

1. Ideation
2. Design
3. Include custom interaction

10. Paper prototypes

1. W1 – W4: User Interface – Rebeca
 1. Home page for unauthenticated users + User authentication page
 2. New user registration page
 3. "Authenticated" home page for regular users. Dashboard of relevant information.
 4. "Analysis" page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service
 5. Analysis page of the user's current data
 6. History screen to show past transactions
 7. Reporting functionality
2. Tabs for admin functionality: View all users, Disable user accounts, Reset user passwords, View 'system' transactions, Perform some sort of analysis across all accounts

Project Requirements – Gitlab and Coding

Legend:

- Done
- In progress
- To be done

11. Organization

1. W2: Setup the overall project repo structure – Rebeca
2. Keep the issues on GitLab up-to-date

12. Full Stack

1. Front-end design
 1. CSS for styling
 2. HTML for all the page prototypes (for the pages that need data, use dummy data)
 3. HTML tabs for admin functionality: View all users, Disable user accounts, Reset user passwords, View 'system' transactions, Perform some sort of analysis across all accounts
2. Back-end design:
 1. JavaScript to connect front end with the API
 2. Data base with user roles and permissions
 3. OAuth2 Authentication Integration
3. Database design:
 1. for user (PostgreSQL)
4. Tests suite (including DFD created)
 1. Unit tests
5. Logging and Monitoring
 1. Understand
 2. Design
 3. Implement

13. CI/CD – YAML file

1. Build release software on every commit
2. Runs test suite on every commit
3. Runs Gitlab SAST Tooling
4. Automatically deploy your software to your VCM

14. Data Service Design

1. W2: API to connect data service with Polymarket (Data Integration using a REST/JSON web API) - Mike
2. API to connect full stack with data service
3. W2: Message queue - Mike
4. W2: Database (PostgreSQL) implementation - Mike
5. Tests suite; also use DFD created

15. Containerize Application

1. Create docker files to create an image of the application
2. Create a docker compose file to deploy and execute image

16. Security Analysis – run tests against the software

1. Run Semgrep analyzer against your code base
2. Runs OWASP Zed Attack Proxy

Project Issues

(updated on March 19)

Link to the issue page: https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform/-/issues?sort=created_asc&state=opened&last_page_size=20&page_before=eyJjcmVhdGVkX2F0IjoiMjAyNi0wMy0xOSAxMDozNTozMS41MDU4MzcxMDAgLTA0MDAiLCJpZCI6IjQ1NTQyIn0

📁 Organization

team-hotel/pred_market_data_platform#21 · created 5 hours ago by yinglun.song@duke.edu

updated 18 minutes ago

✓ List the requirements

team-hotel/pred_market_data_platform#22 · created 5 hours ago by yinglun.song@duke.edu Done

updated 26 seconds ago

✓ Refine the requirements

team-hotel/pred_market_data_platform#23 · created 5 hours ago by yinglun.song@duke.edu Current in Progress



updated 3 minutes ago

📁 Gitlab

team-hotel/pred_market_data_platform#24 · created 5 hours ago by yinglun.song@duke.edu

✓ Create Gitlab

team-hotel/pred_market_data_platform#25 · created 5 hours ago by yinglun.song@duke.edu Done

updated 14 seconds ago

✓ Update the issues (+ improve wording)

team-hotel/pred_market_data_platform#26 · created 5 hours ago by yinglun.song@duke.edu Current in Progress



updated 3 minutes ago

✓ Understand Gitlab SAST Tooling

team-hotel/pred_market_data_platform#27 · created 5 hours ago by yinglun.song@duke.edu Current in Progress



updated 2 minutes ago

📁 System Architecture diagrams (C4)

team-hotel/pred_market_data_platform#28 · created 5 hours ago by yinglun.song@duke.edu Current in Progress



updated 2 minutes ago

✓ Create System Architecture

team-hotel/pred_market_data_platform#29 · created 5 hours ago by yinglun.song@duke.edu Done

updated just now

✓ Create using C4 model

team-hotel/pred_market_data_platform#30 · created 5 hours ago by yinglun.song@duke.edu Current in Progress

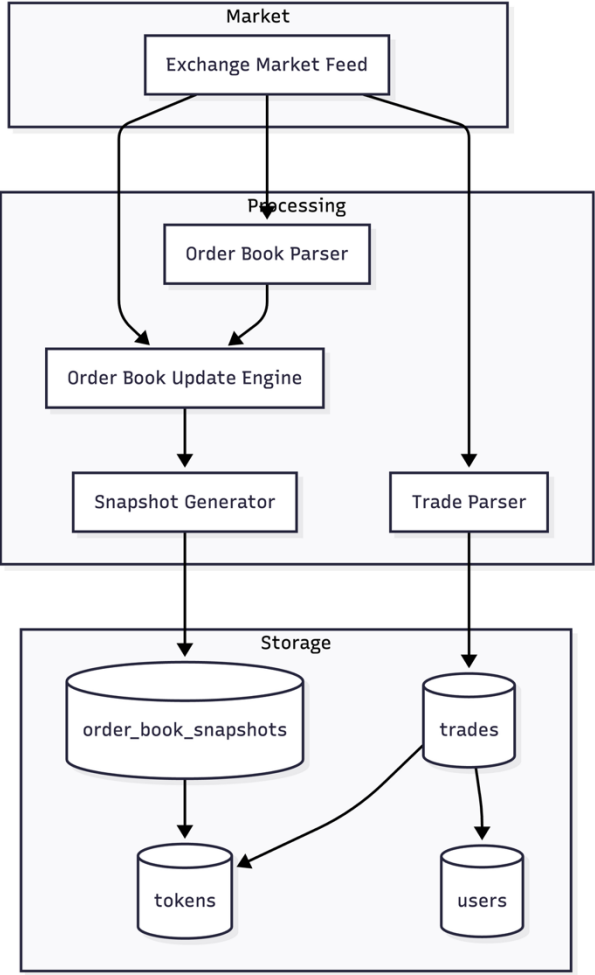


updated 2 minutes ago

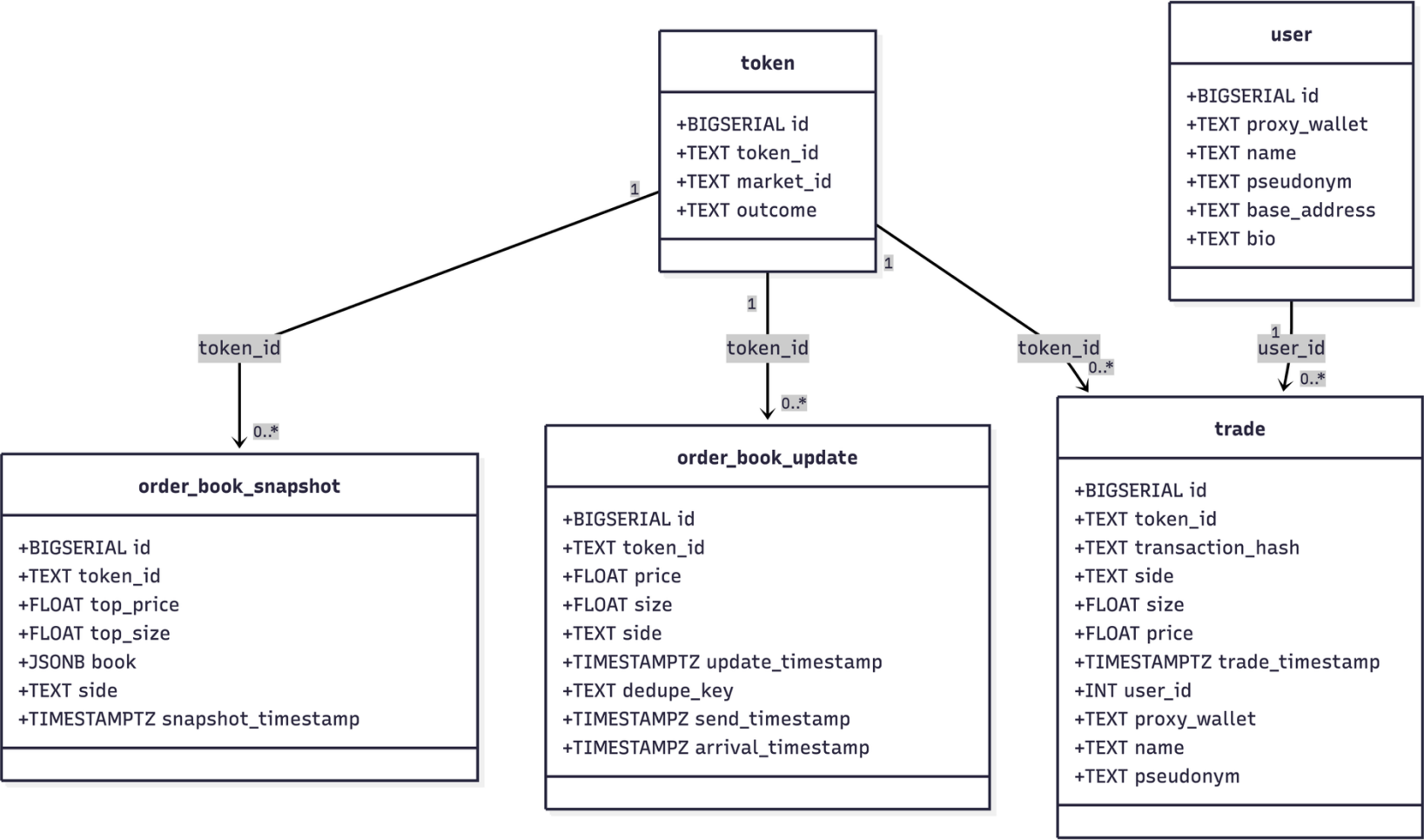
*Some of the issues that are created

Updated Design

UML Class Diagram – Order Book

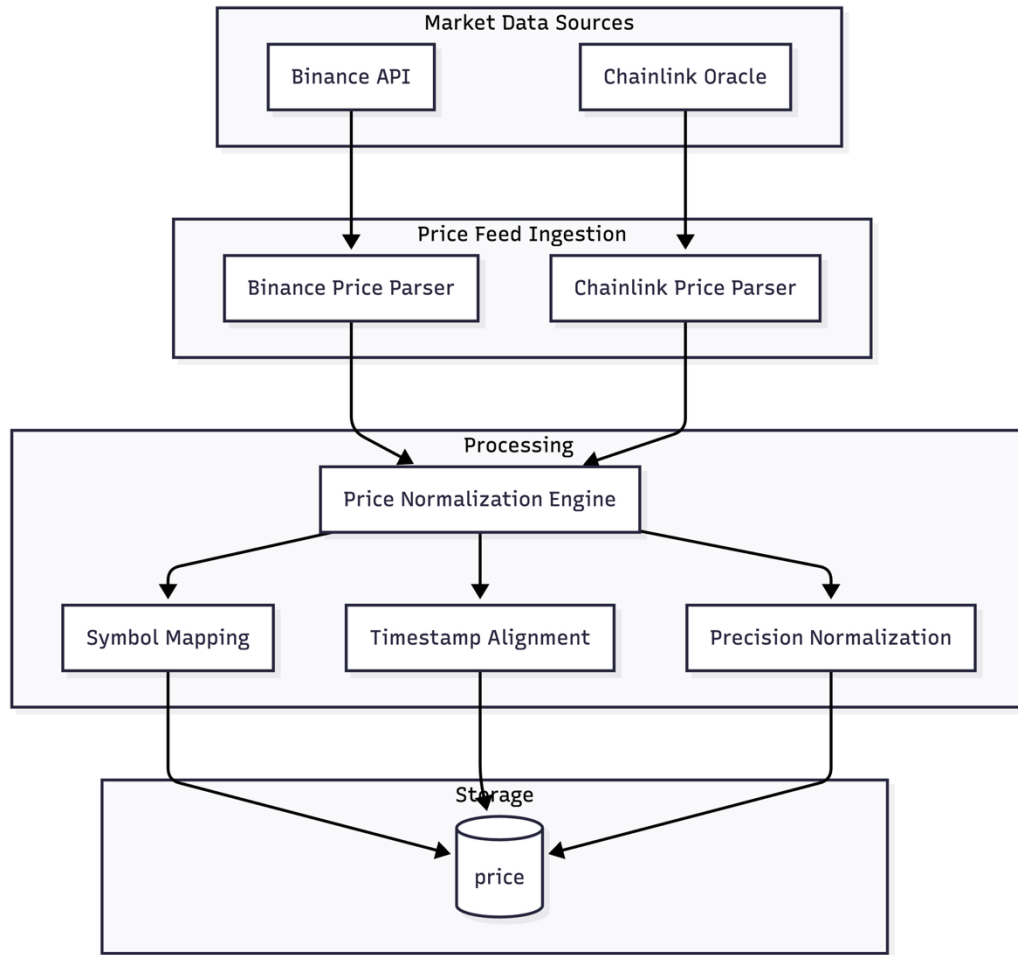


Architecture/pipeline

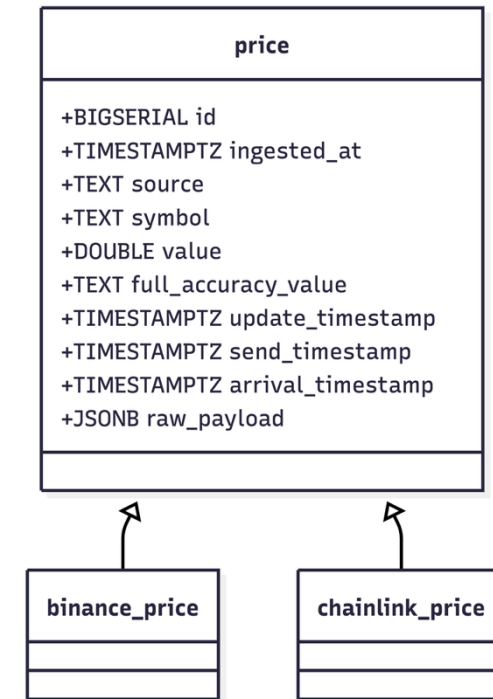


UML Class Diagram

UML Class Diagram - Price

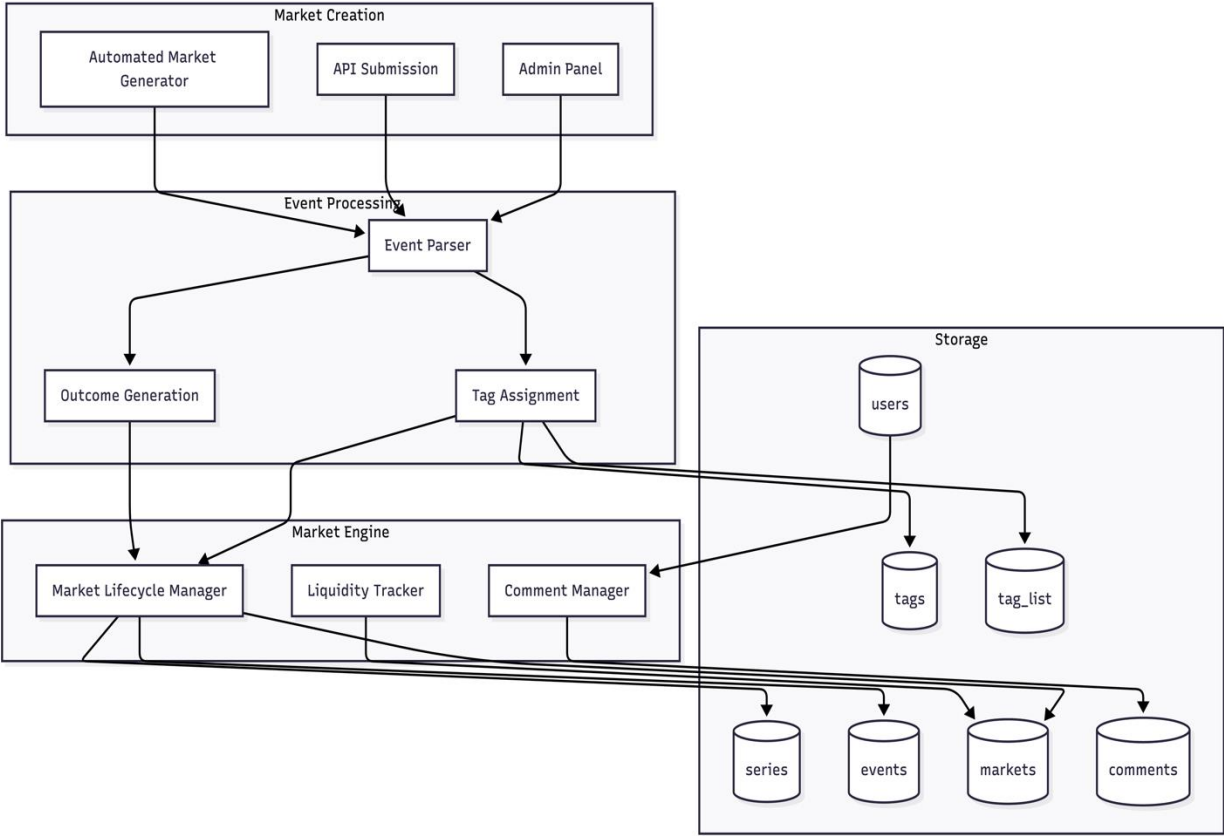


Architecture/pipeline

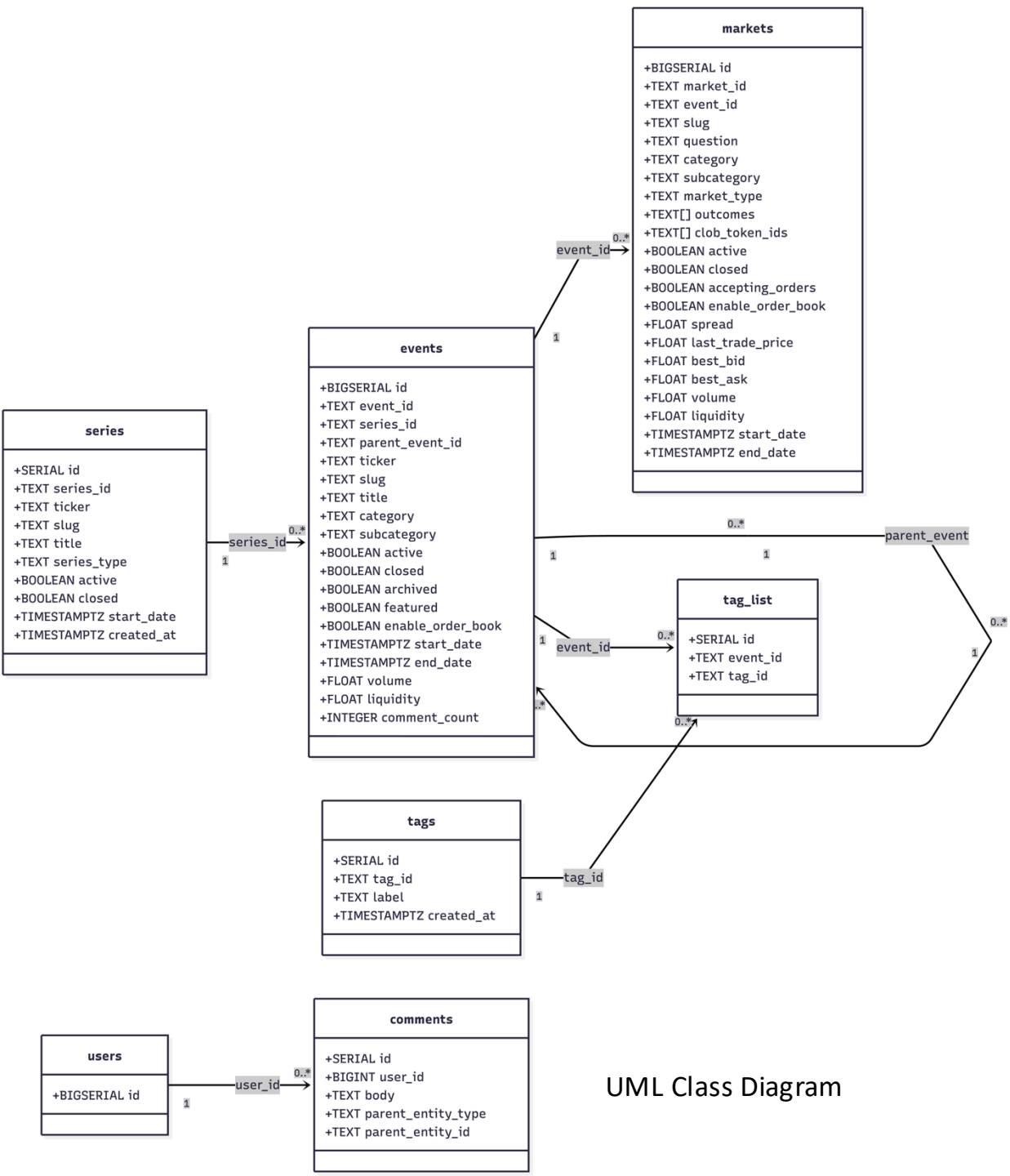


UML Class Diagram

UML Class Diagram - Market



Architecture/pipeline



UML Class Diagram

Architecture

2026/03/06 Design V1

Data source: Polymarket.

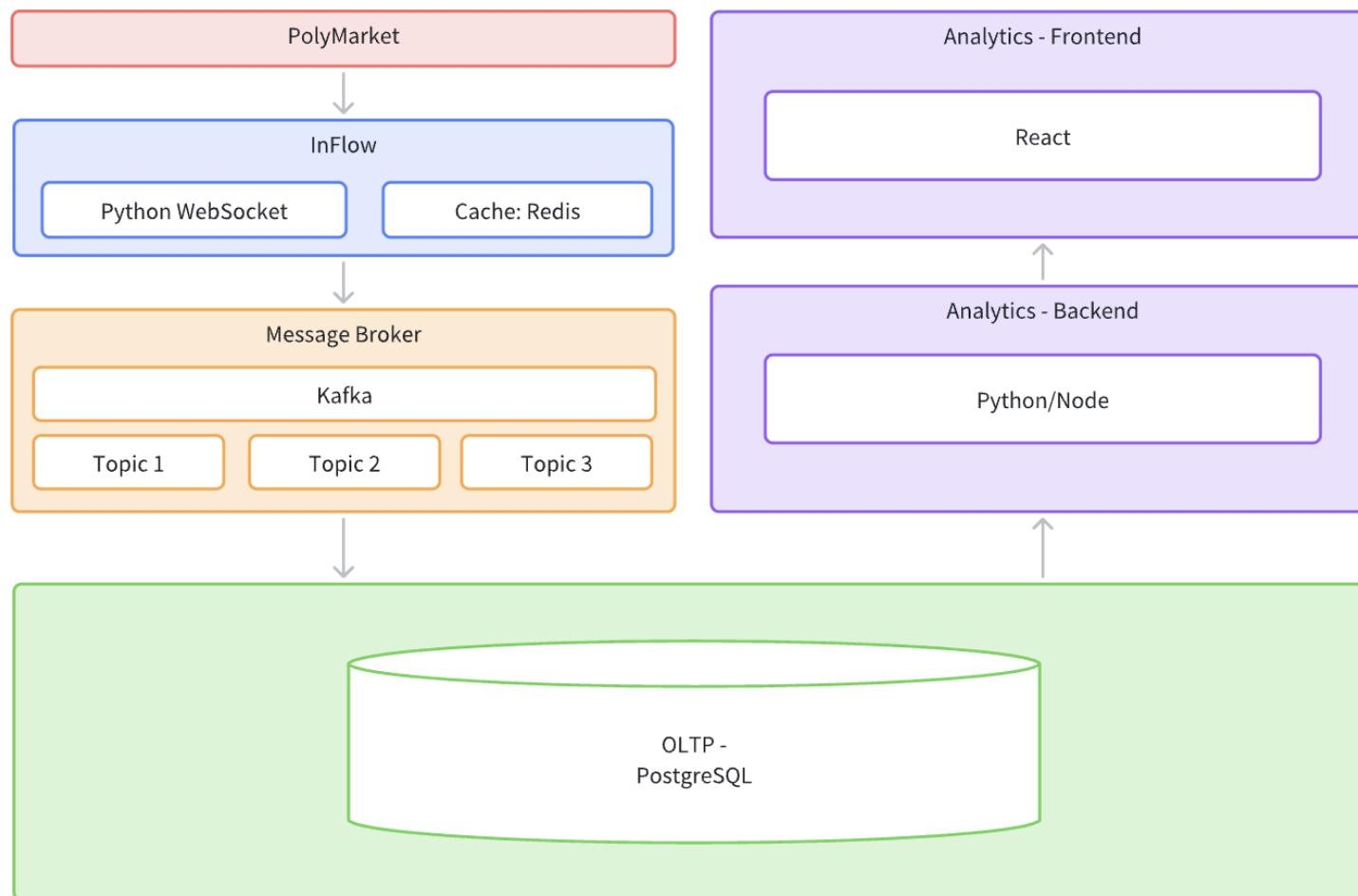
Ingestion layer: Python WebSocket collector with Redis cache.

Streaming backbone: Kafka message broker with multiple topics.

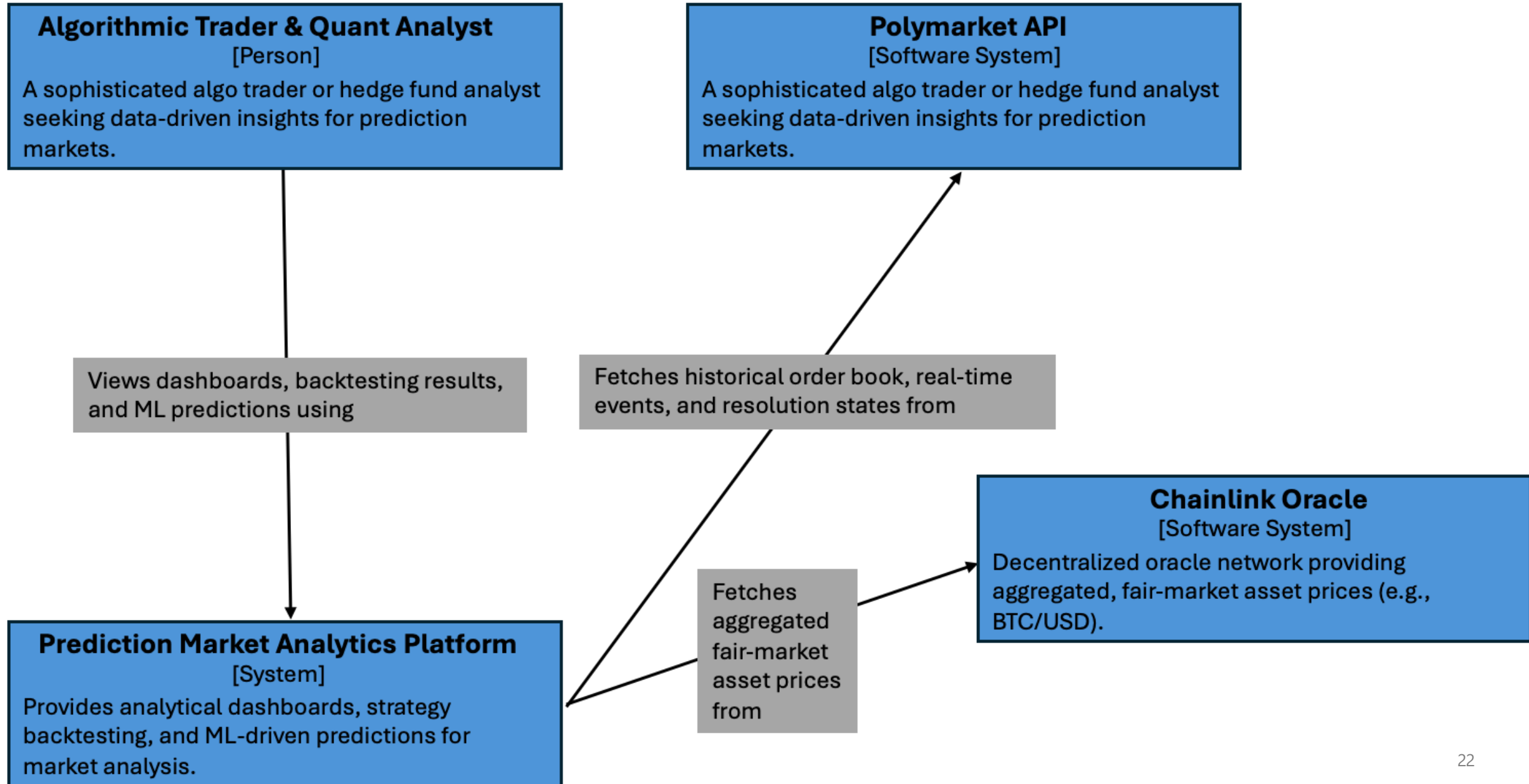
Storage: PostgreSQL (OLTP) for persisted normalized records.

Analytics backend: Python/Node services querying and aggregating stored data.

Analytics frontend: React UI for dashboards and research access.



System Architecture (C4 model) - Level 1 Context

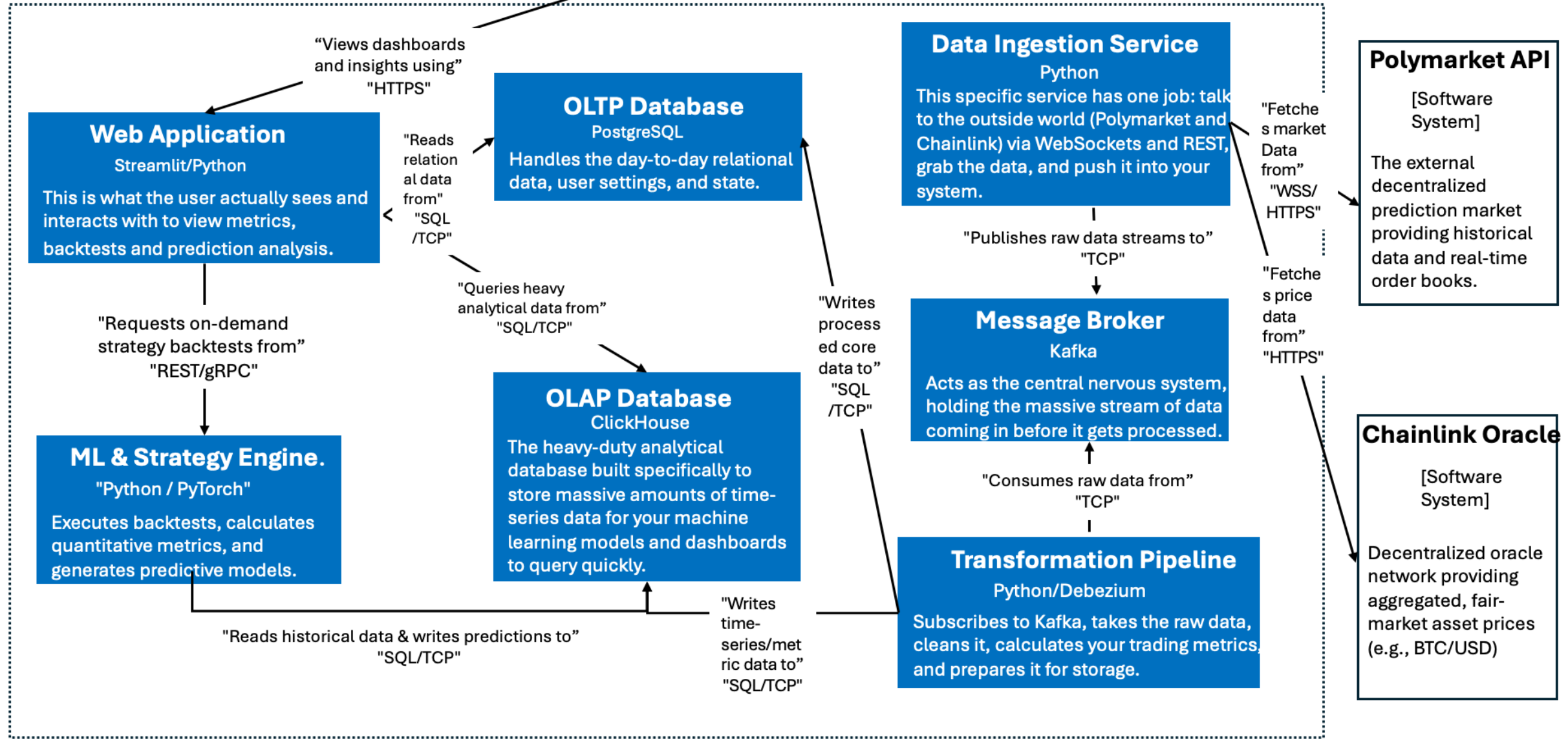


System Architecture (C4 model) - Level 2 Container

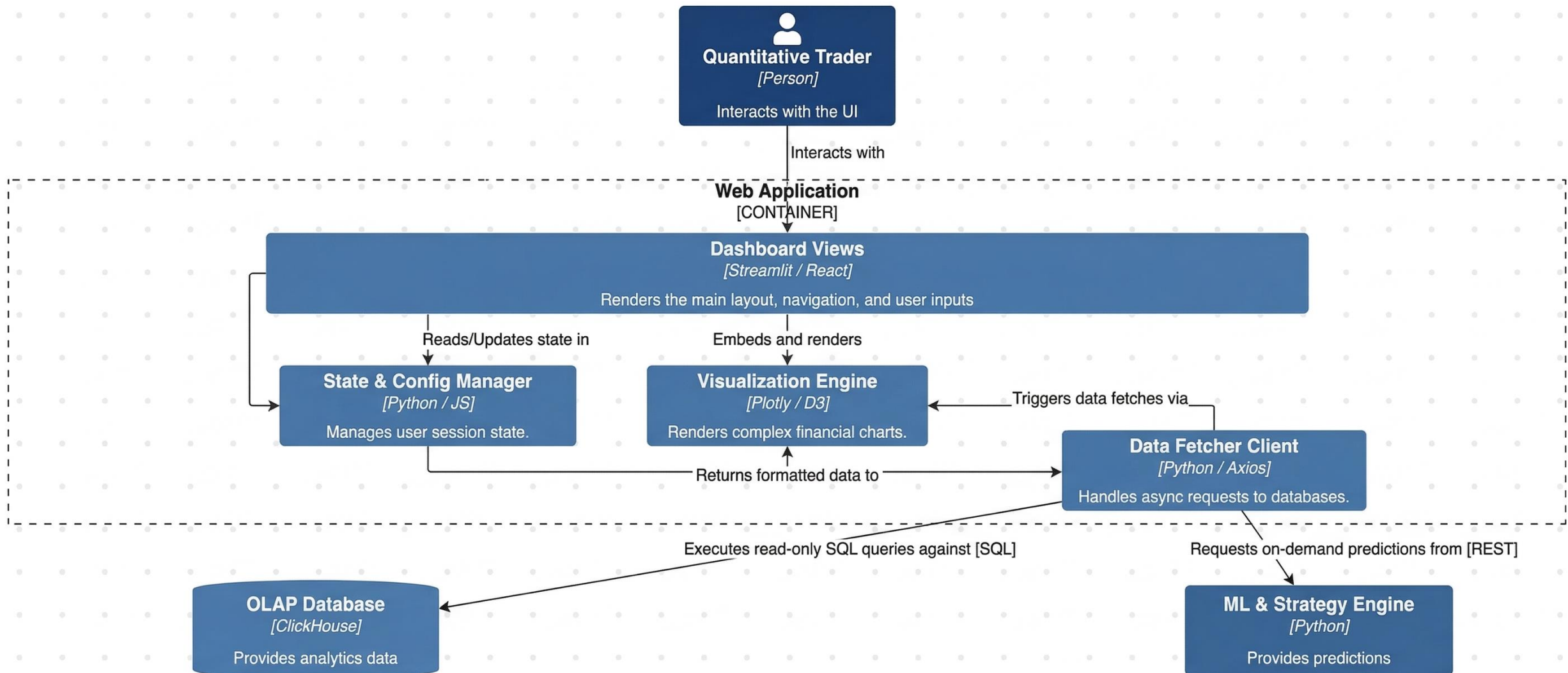
Algorithmic Trader & Quant Analyst

[Person]

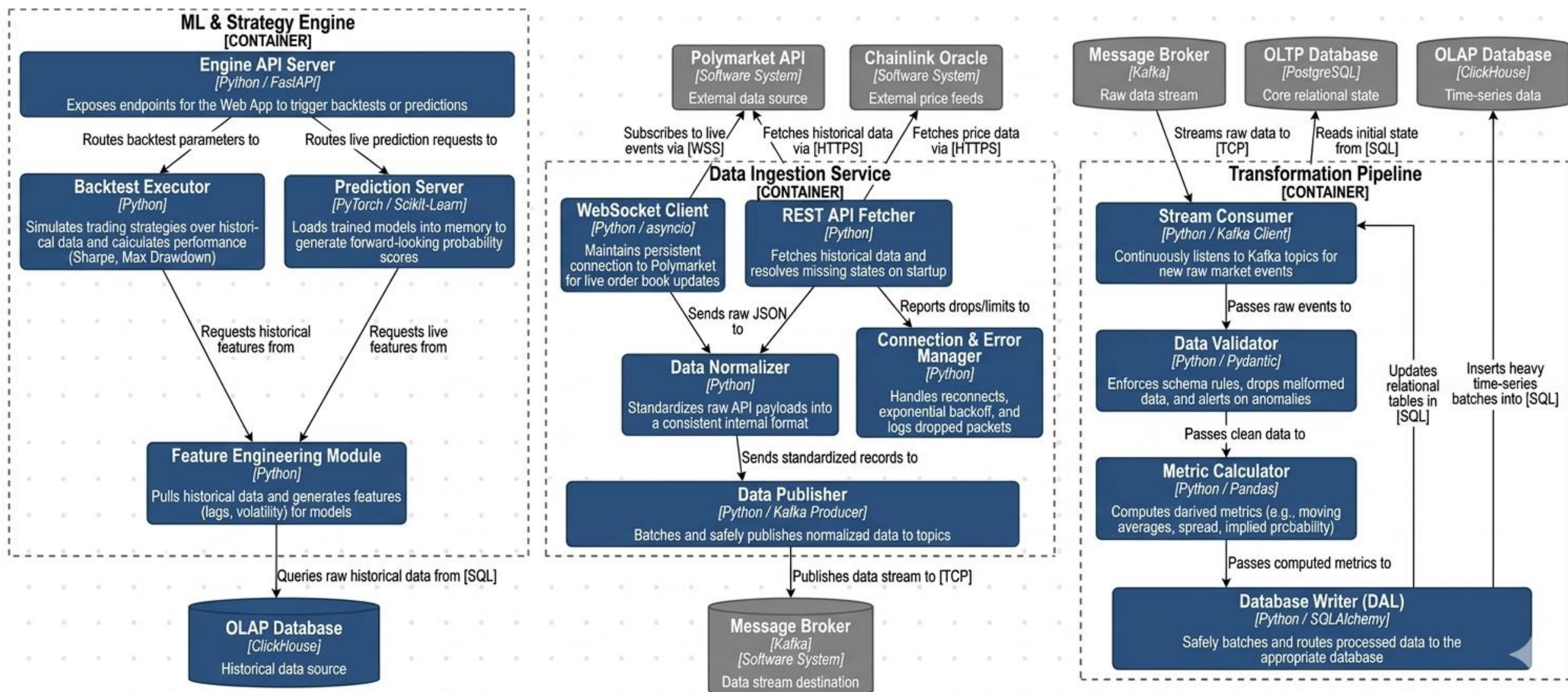
A sophisticated algo trader or hedge fund analyst seeking data-driven insights for prediction markets



Level 3 Component – Web Application



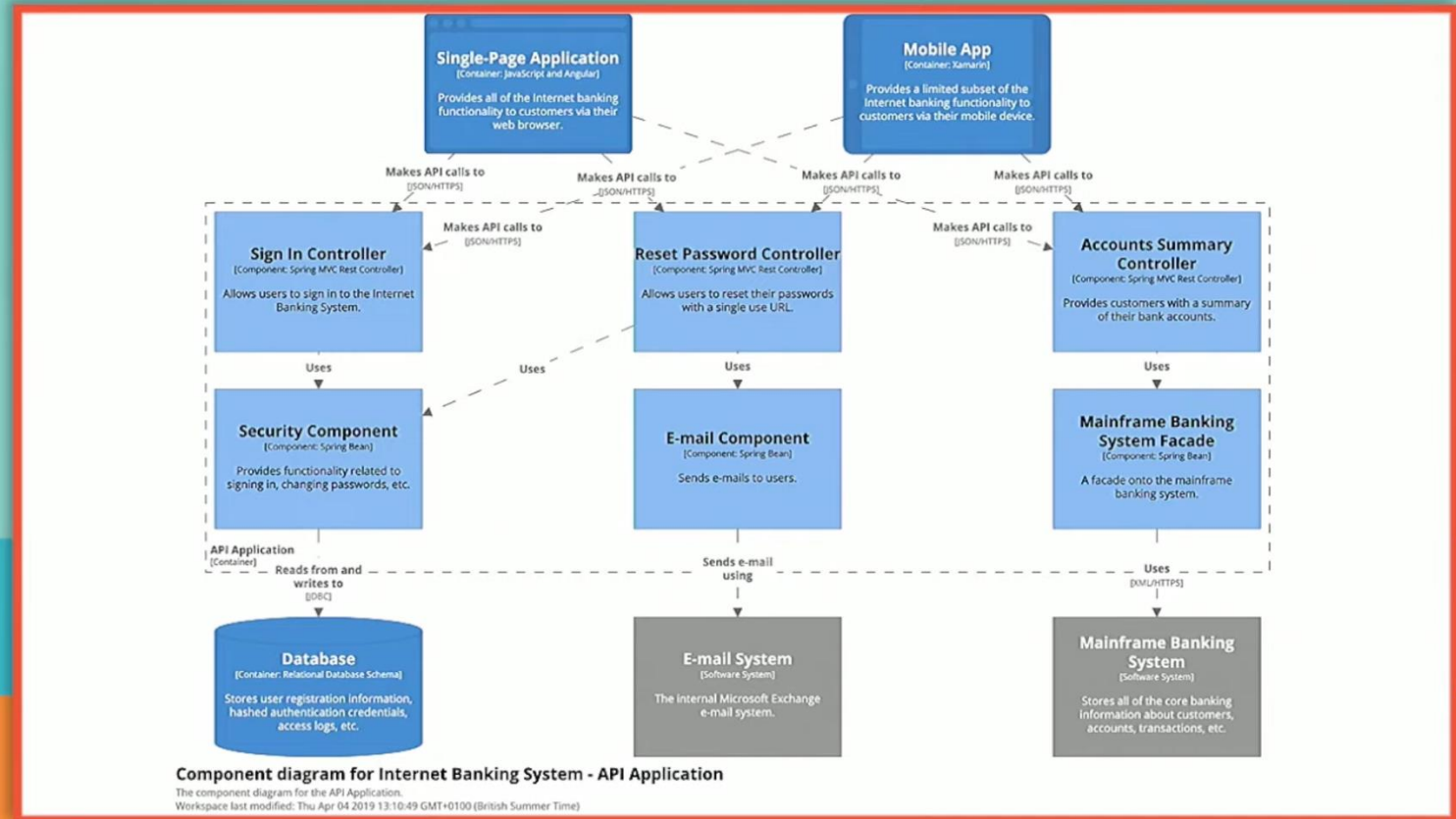
Level 3 Component – The Data Ingestion & Processing Containers



Level 4 Code



AGILE ON THE BEACH



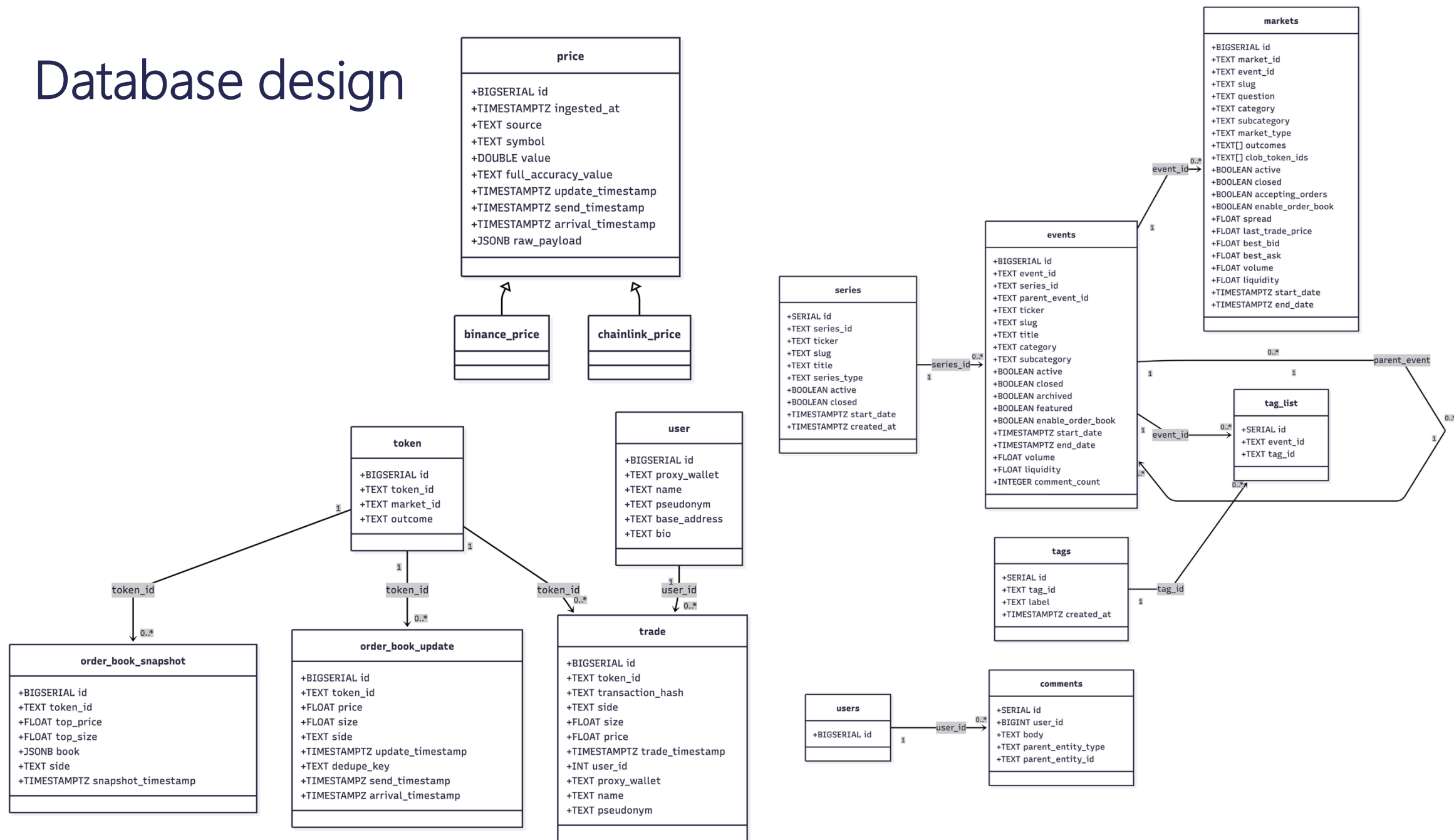
Streaming
sponsored by:

FALMOUTH
UNIVERSITY

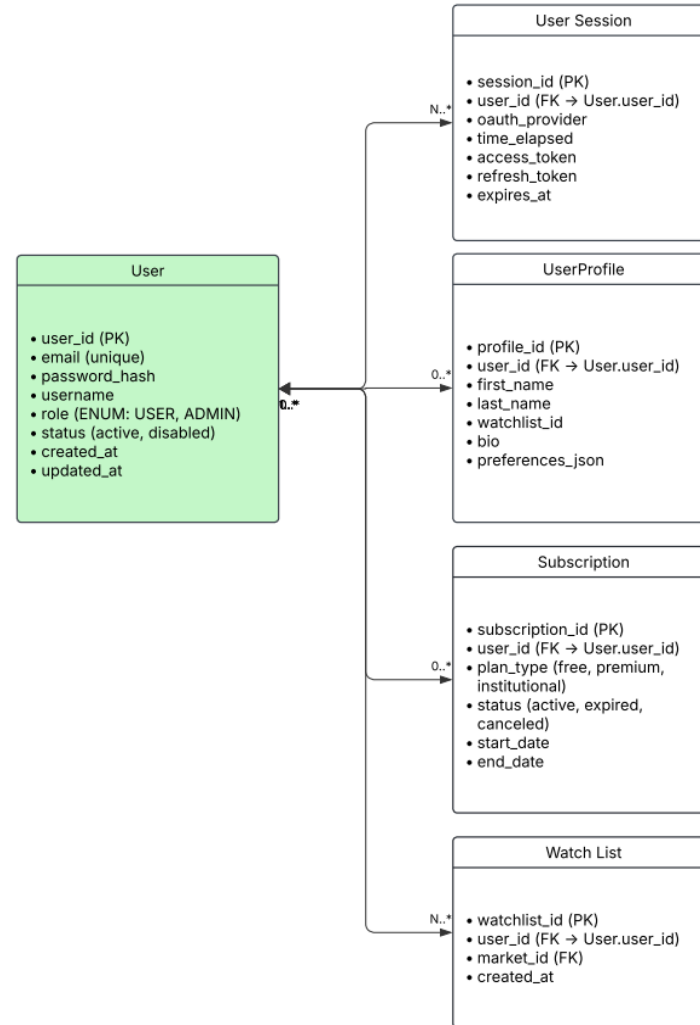
FLEXIBLE
LEARNING

always miss this I do not recommend
doing level 4 because it's not worth it

Database design



Database schema for users



Testing approach Methodology

Unit Tests (The Foundation):

Testing individual functions in isolation.

Task: Does this specific Python function correctly parse a Polymarket probability format into a float?

Integration Tests (The Middle):

Testing whether decoupled system modules communicate correctly.

Task: Does the WebSocket ingestion service correctly write data to the cache?
Does the message broker correctly ingest and forward the event?

Data Quality & Contract Tests (Pipeline Specific):

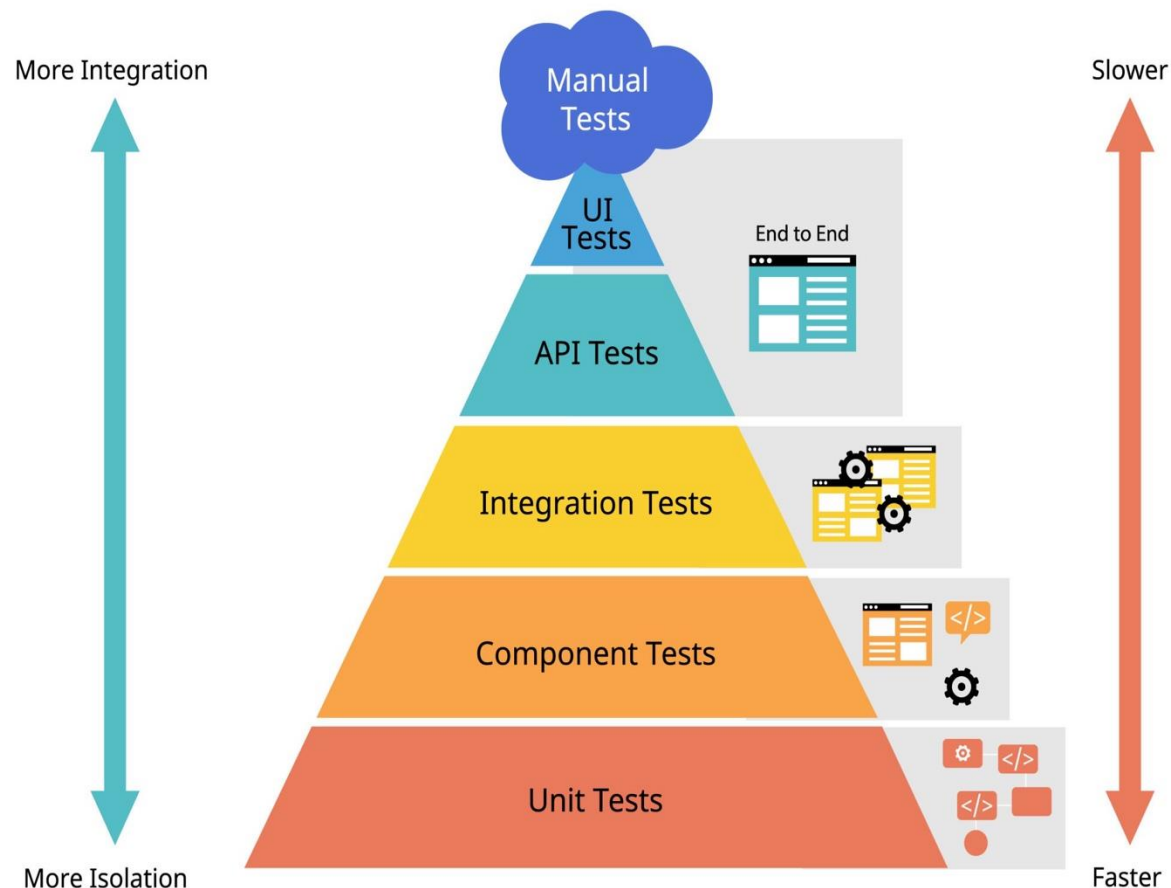
Testing whether schema or upstream data changes are handled safely.

Task: If Polymarket adds a new field to their JSON response, does our pipeline safely ignore it or crash?

End-to-End (E2E) Tests (The Peak):

Testing whether the full system works correctly from input to final user-visible output.

Task: If a trade event happens on the mock Polymarket WebSocket, does it successfully propagate through the pipeline, get stored in OLAP database, and appear correctly on the dashboard?



Testing approach In Data Ingestion

Corner Cases

Network Jitter & Disconnects.

What happens if the Polymarket WebSocket drops packets or silently disconnects? Does the system recognize the stale state?

Malformed Payloads.

Polymarket sends an unexpected null value for a betting pool size, or a string instead of an integer.

Redis Memory Eviction.

If the downstream pipeline slows down and Redis fills up, do we drop new data or block the WebSocket?

Examples from unit test to E2E test

Data Parsing

Objective: Verify that raw JSON from Polymarket is correctly transformed into our internal Python Data Class / Pydantic model.

Implementation: We do not need an internet connection for this. We save a sample Polymarket JSON response as a local fixture file. We write a pytest that feeds this static JSON into our parser function.

Python InFlow to Redis

Objective: Verify that parsed data is successfully cached in Redis with the correct Time-To-Live (TTL).

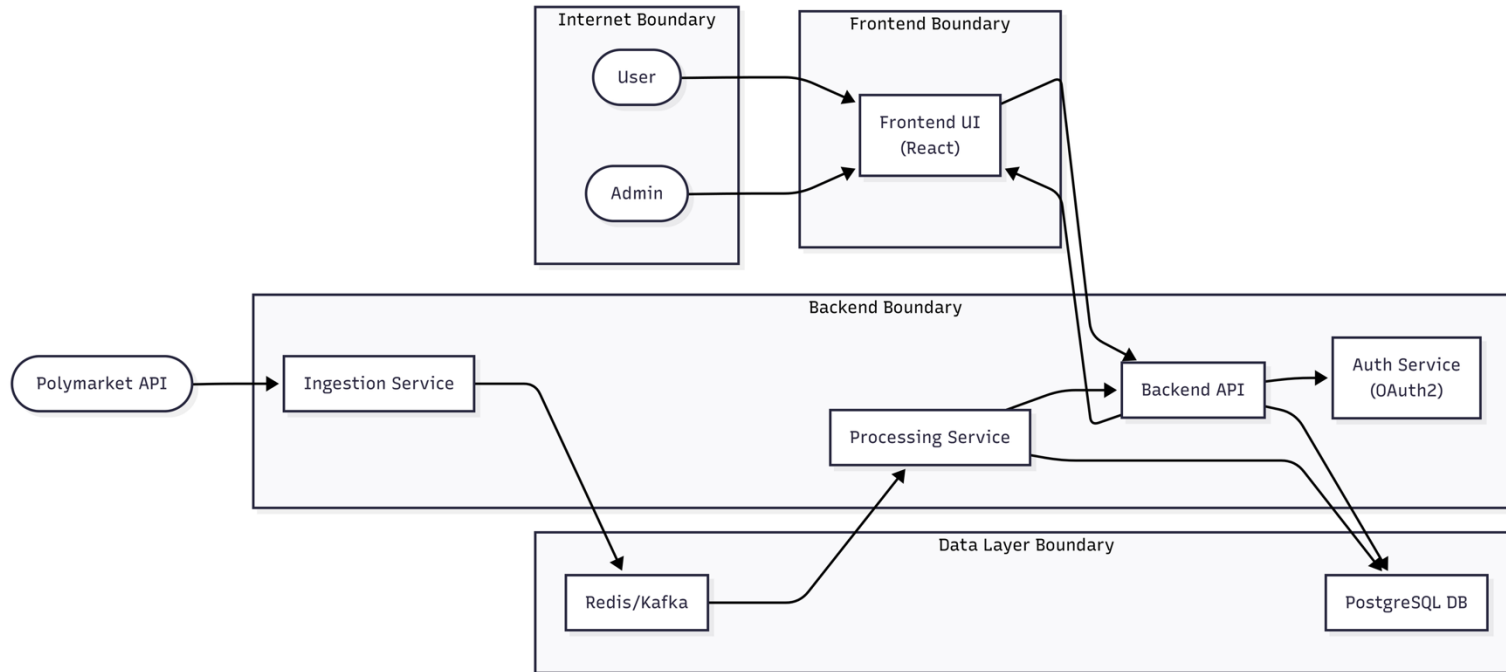
Implementation: We use Testcontainers (a framework that spins up a lightweight, throwaway Docker container of Redis just for the test). Our Python code connects to this temporary Redis instance, writes a mock message, and reads it back.

The Ingestion Slice

Objective: Verify the entire Inflow module works from WebSocket to Message Broker.

Implementation: We create a mock WebSocket server that mimics Polymarket and emits pre-determined trade events at a specific rate. We spin up our Python InFlow service, a Redis instance, and a single Kafka node using Docker Compose.

Data Flow Diagram (DFD)



- **Assets:** User credentials and OAuth tokens, user permissions, Orderbook data and trade records, market structure data, internal APIs and service endpoints, PostgreSQL database contents, external market data (Polymarket based), system logs, audit records.
- **Actors:** User, Administrator, Polymarket API, Backend Services (for handling logics and data processing)
- **Threats:** Based on STRIDE analysis

System Overview:

- Users interact with the system through frontend connected to backend APIs
- Backend services process authentication, logics and provide database access
- External market data – Polymarket is ingested, processed and stored
- Data views as dashboards and analytics components.

STRIDE analysis:

- **Spoofing**
 - Stolen session or OAuth token, external data source is not correlated.
- **Tampering**
 - Modified API requests, data corruption and wrong market data.
- **Repudiation**
 - Users/admin deny actions, missing logs for audit.
- **Information disclosure**
 - Data leakage like user information and API keys, and overexposed admin endpoints.
- **Denial of service**
 - Data pipeline may overload.
- **Elevation of privilege**
 - Users access admin endpoints, and lack of authorization check.

Testing approach - Unit test suite for Full Stack

Corner Cases

Invalid User Inputs. (frontend validation)

A user submits invalid format of forms, registration info, or empty data. The system needs to prevent submission and provide clear validation feedback in the UI level.

Unauthorized Access. (security)

A user tries to access admin routes or attempt to get access to API keys. Access control should timely focus on across both frontend and backend (render and authorization).

API Failure, Timeout or Error Response.

The backend may return an error or slow responses during user requests. The frontend should handle those scenarios without crashing or exposing internal errors.

Invalid Request, Malformed Payload (backend validation)

Users' requests may contain missing fields or incorrect data types or values. The backend should validate the inputs and reject those request correctly.

Examples

Frontend Component Validation

Objective: Verify UI components correctly validate the user input.

Implementation: We render components (login/registration) using testing libraries, simulate possible user inputs (forms/types), and verify validation behavior (without real APIs); thus, the test remain isolated at the component level.

API Helper Function (Frontend)

Objective: Verify that the frontend API utilities construct and handle the requests correctly from backend without too much lagging

Implementation: We monitor backend responses without calling a live server, and test the formats of requests, success handling and error states. We then verify the successful responses or error messages are correctly shown.

Backend Authorization

Objective: Verify protected routes enforce role-based access controls.

Implementation: We simulate authentication tokens and user roles, then test if the endpoints returning the responses correctly. Such as 401 Unauthorized for missing or invalid authentication and 403 Forbidden for insufficient privileges.

Backend Logic with interaction of Database

Objective: Verify if the logic independently of the database, including validating request payloads, transforming input data, returning expected outputs and failure case handling.

Implementation: We call on the database and test if the controller or service functions ensures the correct outputs and the error handling. For example, we simulate correct reads/writes, missing records or database exceptions.

Testing approach - Unit test suite for Data Service

For Inflow Gateway, we use a strong mocking setup for unit tests.

- pytest monkeypatch
- custom test doubles
- fake WebSocket connections
- recording publishers
- fake HTTP responses

On the right are a few corner cases, and how we test for them.

1. External dependency failures

- WebSocket API unavailable
- Redis unavailable
- network I/O interruption

2. Data processing edge cases

- incorrect or unexpected data transformation
- invalid market filtering
- missing or incomplete payload enrichment

3. Resilience and recovery scenarios

- timeout during data fetch
- WebSocket connection failure
- retry behavior after failure
- graceful degradation when a dependency breaks

4. Data quality and integrity issues

- duplicate message detection
- malformed or partial payloads
- incorrect publish behavior

Class design

Trading/Order Book System

- Core class: token, order_book_snapshot, order_book_update, trade, user
- Relations:
 - Token -> order_book_snapshot
 - Token -> order_book_update
 - Token -> trade
 - User -> trade

Price Oracle System

- Core class: price, binance_price, chainlink_price
- Relations: price --> binance_price / chainlink_price

Market Structure System

- Core class: series, events, markets, tags, tag_list, comments, users
- Relations:
 - series -> events -> markets
 - events -> events (parent_event)
 - events -> tag_list -> tags
 - users -> comments

Usage

Store the order book state and trade executions for each token.

Provides external reference prices from different oracle sources.

Defines the hierarchical structure of prediction markets and their metadata.

APIs

External Polymarket Gamma REST endpoints documented:

- `GET /events`
- `GET /series`
- `GET /trades`

Internal persistence endpoint:

- `POST /internal/postgres/write`
- Allowed table targets:
- `chainlink\_prices`
- `binance\_prices`
- `kafka\_price\_changes`

Please refer to api.yaml file in the git repository for additional details.

Polymarket Gamma API 1.0.0 OAS 3.0

External API for Polymarket market data.

Servers

https://gamma-api.polymarket.com - REST API server

Filter by tag

default

GET /events Fetch market events

GET /series Fetch market series

GET /trades Fetch trade history

POST /internal/postgres/write Batch write to Postgres

Schemas

GammaEvent >

Error handling

Domain & Logic Safeguards (Task-Driven)

Malformed Ingestion Payload (Schema Changes)

Mechanism: Dead Letter Queue (DLQ). Catch `ValidationError`, route bad data to a separate Kafka topic, and skip to the next message without crashing the pipeline.

Quantitative Calculation Exceptions (e.g., Missing Data, `ZeroDivisionError`):

Mechanism: Graceful Degradation. Wrap core math in try-except blocks. Output `stale_data` flags or safe previous values to prevent NaN or Infinity from polluting the Clickhouse OLAP.

Infrastructure & System Safeguard (NFR-Driven)

Downstream Service Timeout (>500ms API/DB response)

Mechanism: Circuit Breaker Pattern. Temporarily halt requests to the failing service to prevent cascading failures; serve historical fallback data from the Redis cache.

Kafka/WebSocket Connection Drops

Mechanism: Exponential Backoff with Jitter. Catch `ConnectionError` and reconnect with increasing, randomized delays (e.g., 1s, 2s, 4s + random ms) to avoid network storming.

Spike in Trade Volume (Consumer Lag)

Mechanism: Backpressure & Auto-Scaling. Monitor Kafka consumer lag continuously. If thresholds are breached, trigger AWS ECS to dynamically spin up additional Python processing containers.

Non-Functional Requirements

Real-Time Performance

Low-latency ingestion and processing for live Polymarket data, enabling timely analytics and strategy response.

Reliability & Fault Tolerance

Automatic retry, reconnection, buffering, and graceful degradation to keep the pipeline running under partial failures.

Scalability

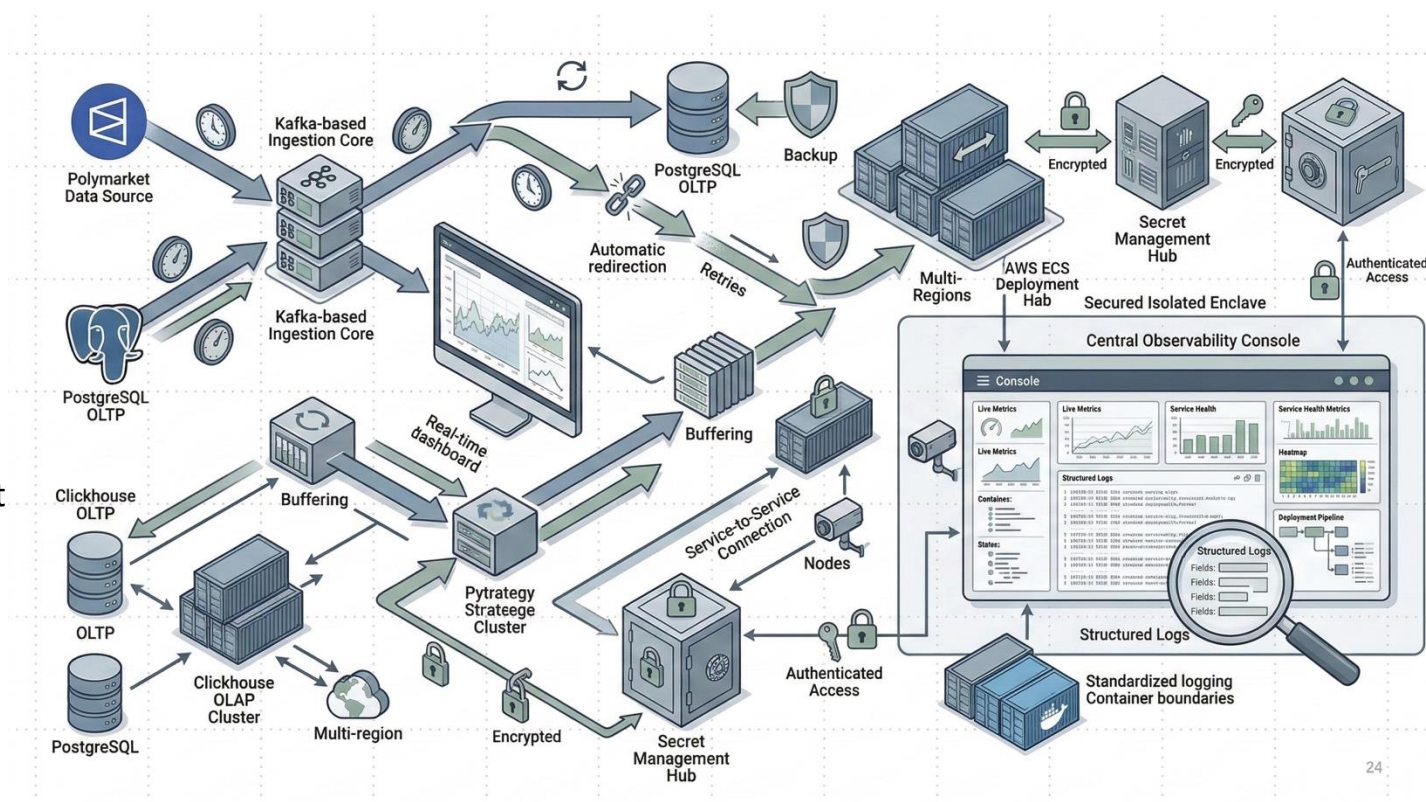
Modular services and message-driven architecture allow independent horizontal scaling as market volume grows.

Security & Access Control

Secure secret management, authenticated service access, and environment isolation to protect system resources and credentials.

Maintainability & Observability

Clear module boundaries, containerized deployment, structured logging, and monitoring to simplify debugging, testing, and future extension.



Status, Review and Planning

TA Discussion and Weekly Meetings Scheduled

We successfully met with our assigned TA last Friday to review our project scope and requirements

Next meeting with Dongjie Ma on March 20
(weekly meetings on Friday at 12:30)

The screenshot shows a Microsoft Teams meeting in progress. The video feed at the top displays three participants: Dongjie Ma, Loki Ziye, and another person. Below the video, a browser window is open to the Duke University course page for 'Spring 2024'. The page includes a sidebar with navigation links like Home, Announcements, Syllabus, Schedule, Assignments, Grades, Project Backlog, and Zoom. The main content area shows a 'Criteria' table with columns for 'Criteria', 'Ratings', and 'Pts'. The table lists various project requirements and their corresponding ratings and points.

Criteria	Ratings	Pts
Team Project GitHub Repository Repo for team project has been created	10 pts Full Marks	0 pts No Marks
Project Requirements 1. Create a lean canvas document for your project. 2. Develop two or more paper prototype screens to highlight the significant functionality of your application. 3. Discuss your project with your assigned TA	15 pts Full Marks	0 pts No Marks
Project Backlog Create a set of GitHub issues representing the requirements you've identified for the project.	10 pts Full Marks	0 pts No Marks
Initial Design Perform the initial design for your application. Items to consider: Architecture Class design Database design	10 pts Full Marks	0 pts No Marks
APIs Testing approach Error handling Implementing necessary non-functional requirements (security/performance)	10 pts Full Marks	0 pts No Marks
Weekly Meetings Scheduled Schedule your team's weekly meetings with your assigned TA.	5 pts Full Marks	0 pts No Marks
Status Report Each project team must complete a sprint status report at the end of each spring with the following elements: A list of the GitHub issues opened or closed during the week. UML class diagram (The first report will be the most difficult; later reports will need to update the diagram.) A paragraph summarizing your sprint review A paragraph summarizing your sprint retrospective A paragraph summarizing your sprint planning Any additional comments and/or concerns	15 pts Full Marks	0 pts No Marks

The screenshot shows a Microsoft Teams calendar interface. The main view displays a meeting titled '512 - Weekly Meetings - Team Hotel' scheduled for Friday, March 20, 2026, from 12:30 PM to 1:00 PM. The meeting is organized by Dongjie Ma and includes attendees Mory Shi, Haoan Lin, Loki Ziye, Yinglun Song, and Rebeca Hechtman. The meeting is marked as 'Optional'. The interface also shows a 'Scheduling Assistant' button and a 'Join Teams meeting' button. The meeting details include the meeting ID (246 502 053 882 81) and the passcode (v3T7hC7e).

Sprint Review Summary

During the second sprint, our team continued to expand both the product design and technical foundation through iterative development and refinement.

We introduced new prototypes to further clarify user interactions and improve the overall experience, and also refining the project requirements to ensure alignment within product vision.

On the technical side, we established a clear directory structure to support scalable development and maintain code organizations. We initiated early-stage system analysis by drafting DFD and conducting a preliminary STRIDE threat modeling to identify potential security risk. Additionally, we defined our testing strategy, focusing on building unit test suites for both the full stack and the data service components to ensure system reliability.

In addition, we developed C4 system architecture model to better visualize system components and iterations. Project management was strengthened through continuous issue tracking and updates in Gitlab, for more structured progress checks. Finally, we designed the initial database schema for user management, laying the groundwork for backend development and data processing and handling.

Sprint Retrospective Summary

Looking back at Sprint 2, the team collaborated effectively to bridge the gap between business requirements and technical design. Our communication was strong, particularly when brainstorming the project and flow. However, we also observed that some technical alignment—particularly around API structures, data ingestion design, and the testing strategy—took longer than expected. These discussions required deeper coordination because multiple components of the system depend on them.

Also, for delegating specific tasks and assigning groups, we spend much time on analyzing the questions and individual strength, which is time-consuming. Nevertheless, we believe the time spent is well-deserved for it is valuable in smoothing the process along the project's progress.

For Sprint 3, we plan to improve efficiency by breaking complex architectural decisions into smaller discussion topics earlier in the week and assigning clearer ownership for technical components. This will help the team reach consensus faster and maintain steady development progress. Moreover, it allows each team member to exercise their individual strength to the fullest.

Sprint Planning Summary

For Sprint 3, our objective shifts toward continuous deployment, comprehensive testing, and finalizing our user interface designs. We plan to tackle the advanced integration and frontend deliverables in our project backlog. Key deliverables for the upcoming sprint include:

- 1. UI Prototyping:** Finish the comprehensive paper prototypes for all UI core pages (authentication, analysis dashboards); Start the design of the admin tabs
- 2. Charting Design:** Initiate the ideation of market data charts
- 3. Advanced Testing & Security Analysis:** Refine testing ideation for both the full-stack and data services
- 4. Build API** to connect full stack with data service

We have scheduled our recurring weekly TA meeting for Friday 12:30 to ensure we remain aligned with the project constraints and technical requirements.

Additional Comments and/or Concerns

General Comments: The team is aligned and ready to begin executing the technical backlog in the upcoming week.

Risks/Blockers: Currently, there are no major blockers. However, we will be actively monitoring concerns as the ones below:

Risk1 – Operational Overhead: Our approach strongly resembles the microservice architecture, while it allows for better modulization, it also adds maintenance overhead to the team. We need to closely monitor the status of each service, and also added more automated orchestrations such as CICD flows to ensure smooth operations.

Risk2 – API Access Concern: Polymarket may retrieve/change the accessibility to their official API endpoints, and it will significantly impede our development.

Risk 3 – Prediction Model Performance Risk: The performance of the prediction model may not meet the required accuracy or reliability thresholds. If the model fails to generate meaningful predictions, the analytics component may not provide sufficient value for deployment in the production dashboard.

Risk 4 – Massive Data Storage Need: As we are capturing high frequency tick-level orderbook data, the need for storage is overwhelming. We are looking at the need of roughly ~20 GB of storage need per day.

Thank you!